

DOCVAULT

Your Documents. Secured.

Phase 1 — Image to PDF + Camera Capture

Updated Complete Product & Technical Documentation

Phase 1 now includes both Camera Capture and Gallery Image Selection. Users can capture documents or select existing images, review/reorder them, generate a PDF, save it locally, open it and share it.

Product	DocVault
Phase	Phase 1 — Image → PDF + Camera
Platform	Flutter / Dart
Mode	Local / Offline-first
Backend	None required
Inputs	Camera + Gallery
Output	PDF

Phase 1 Goal: Let a user choose Camera or Gallery, create/review a document set, convert it to PDF locally, then open or share the result.

1. Product Overview

- DocVault is a modern document utility app. Phase 1 converts real-world documents and existing images into PDFs.
- Users have two input sources: Camera for new captures and Gallery for existing images.
- Camera and gallery images can be combined in the same PDF job.
- The core workflow is local/offline-first; no account, Firebase or online conversion API is required.

2. Phase 1 Objectives

- Clean Home screen with Camera and Gallery entry points.
- Capture multiple document photos in one camera session.
- Select multiple existing gallery images.
- Combine camera and gallery images.
- Review, remove, add and reorder images.
- Generate one PDF with one image per page.
- Save PDF locally and provide Open and Share.
- Handle permissions, cancellation and common errors safely.
- Keep the core workflow usable without internet.

3. Phase 1 Scope

Feature	Status	Notes
Home	Included	Create PDF and Camera/Gallery choices.
Camera	Included	Live preview and repeated capture.
Gallery	Included	Single/multiple existing images.
Combine sources	Included	Camera + Gallery in one PDF.
Review	Included	Thumbnails before generation.
Remove/Add/Reorder	Included	Manage PDF page order.
PDF generation	Included	One image per PDF page.
Local save	Included	Persistent local PDF.
Open/Share	Included	Native device actions.
Edge detection	Future	Not automatic in Phase 1.
Auto crop/perspective	Future	Advanced scanner.
OCR	Future	Text extraction.
Cloud backup	Future	Later Firebase/cloud phase.

4. Updated User Flow

- Home → Create PDF → Camera OR Gallery → Review/Reorder → Generate PDF → Result → Open/Share.
- Camera: Home → Camera → Live Preview → Capture Page 1 → Capture Page 2 → ... → Done → Review.
- Gallery: Home → Gallery → Select images → Review.
- Review supports Add More → Camera/Gallery, Remove and Reorder.

- **Example:** capture 4 assignment pages with Camera, add 1 existing image from Gallery, review 5 pages, then generate one PDF.

5. Phase 1 Screens

- **Splash:** DocVault branding while initializing.
- **Home:** Create PDF plus clear Camera and Gallery options; no login.
- **Camera:** Live preview, capture button, multiple captures, Done/Continue.
- **Gallery:** Multi-image picker with safe cancellation.
- **Review:** All camera/gallery images, Add More, Remove, drag reorder and Generate PDF.
- **Generation:** Loading/progress and duplicate-generation protection.
- **Result:** File name, page count where practical, Open, Share and retry/error state.

6. Functional Requirements

ID	Requirement	Acceptance
FR-01	Create PDF	Opens Camera/Gallery choices.
FR-02	Camera permission	Requested when Camera is first used.
FR-03	Camera capture	User can capture document photos.
FR-04	Multiple captures	Multiple pages can be captured in one session.
FR-05	Gallery selection	One or multiple images can be selected.
FR-06	Combine sources	Camera + Gallery can be used together.
FR-07	Review	All images visible before generation.
FR-08	Remove	Any image can be removed.
FR-09	Add More	More camera/gallery images can be added.
FR-10	Reorder	User controls final page order.
FR-11	Generate	Valid PDF is produced.
FR-12	Page mapping	Each image becomes one PDF page.
FR-13	Local save	PDF is persisted locally.
FR-14	Open	PDF opens with compatible viewer.
FR-15	Share	PDF enters native share flow.
FR-16	Permissions/errors	Denied access and failures are handled.
FR-17	Offline	Core conversion works without internet.

7. Non-Functional Requirements

- Responsive camera preview and image selection.
- No crashes on permission denial, cancellation, missing/corrupt files or camera errors.
- Privacy-friendly local processing.
- Offline-first conversion.
- Separate camera, gallery, PDF and storage services.
- Responsive UI during processing.
- Test across supported Android devices.

8. Technical Stack

Layer	Recommended approach
Framework	Flutter + Dart
Camera	camera package or maintained Flutter camera solution
Gallery	image_picker or suitable multi-image picker
PDF	pdf package
PDF/print	printing where appropriate
File paths	path_provider + platform file APIs
Sharing	share_plus
State	One consistent approach: Provider, Riverpod, Bloc/Cubit, etc.
Metadata	Hive/Isar if needed; SharedPreferences only for small settings
Backend	None in Phase 1
API	None for core conversion

9. Recommended Flutter Structure

```
lib/  
  ■■■ main.dart  
  ■■■ core/  
    ■ ■■■ constants/  
    ■ ■■■ theme/  
    ■ ■■■ utils/  
    ■ ■■■ services/  
  ■■■ features/  
    ■ ■■■ home/  
    ■ ■■■ camera/  
      ■ ■ ■■■ screens/  
      ■ ■ ■■■ widgets/  
      ■ ■ ■■■ services/  
    ■ ■■■ image_selection/  
    ■ ■■■ pdf_generation/  
      ■ ■ ■■■ screens/  
      ■ ■ ■■■ services/  
      ■ ■ ■■■ models/  
    ■ ■■■ pdf_result/  
  ■■■ models/
```

10. Camera Integration Requirements

- Open Camera only after the user chooses Camera.
- Request runtime camera permission when required.
- Show live camera preview and a clear shutter button.
- Support repeated capture for multi-page documents.
- Add every captured photo to the same image-selection state used by Gallery.
- Done/Continue returns the user to Review.
- Permission denial must show a clear retry path and Gallery alternative.
- If camera initialization fails or no usable camera exists, show an error and allow Gallery.
- Phase 1 does not include automatic edge detection, perspective correction or auto crop.

11. PDF Generation Logic

The PDF service receives the ordered list of camera/gallery images. It creates one page per image, maintains a reasonable aspect ratio, and writes the resulting PDF to persistent local storage.

Image 1 → Page 1
Image 2 → Page 2
Image 3 → Page 3
...
Image N → Page N

12. Local Storage Design

- Generated PDFs should be saved in a persistent local document/file location appropriate to the Android version.
- Camera processing files may be temporary and should be cleaned when no longer needed.
- Example: App / Documents / PDFs / generated_file.pdf.
- SharedPreferences is not for storing PDF/image files; use real files for documents.

13. Permissions & Android Considerations

- Camera permission is required for Camera.
- Gallery/image access should follow the target Android version and picker package behavior.
- Request permissions contextually, not all at startup.
- Handle denied/permanently denied permissions without crashing.
- Do not request unrelated permissions.
- Use modern Android storage behavior.

14. Error Handling

Situation	Expected behavior
Camera permission denied	Explanation + retry + Gallery alternative.
Camera init fails	Clear error + Gallery alternative.
Camera cancelled	Return safely.
Gallery cancelled	Return safely.
No images	Disable Generate or show message.
Bad image	Show error + remove/retry.
PDF failure	Stop loading + retry.
Save failure	Tell user PDF could not be saved.
No viewer/share	Clear message, no crash.

15. UI/UX Requirements

- Professional, clean document-utility design.
- Camera and Gallery should be equally discoverable on Home.
- Large touch targets and clear icons.
- Camera screen prioritizes preview and capture.
- Review clearly communicates page order.
- Obvious Remove, Add More and Reorder controls.
- Visible generation loading state.
- Responsive design for different Android screen sizes.
- Consistent DocVault branding.

16. Branding

- **App:** DocVault

- **Tagline:** Your Documents. Secured.
- **Concept:** Capture, convert and manage documents in one place.
- The tagline is positioning; Phase 1 should not claim enterprise-grade encryption unless such security is actually implemented.

17. Testing Plan

Test	Expected result
Camera permission allowed	Camera opens.
Permission denied	Clear message + retry/Gallery.
One capture	One image reaches Review.
Multiple captures	All pages reach Review in capture order.
Camera + Gallery	Both sources combine.
Gallery multiple selection	All images appear.
Reorder	PDF follows new order.
Remove	Removed image absent.
Add More	New images included.
Offline	Conversion works without internet.
Open	PDF opens.
Share	PDF enters share flow.
Restart	Saved PDF remains available.

18. MVP Acceptance Criteria

- User can start a new PDF and choose Camera or Gallery.
- Camera permission works correctly.
- User can capture multiple pages.
- User can select multiple gallery images.
- Camera and Gallery images can be combined.
- User can review, remove, add and reorder.
- Valid multi-page PDF is generated in review order.
- PDF is saved locally and can be opened/shared.
- Common errors do not crash the app.
- Core workflow works offline.
- No account/Firebase/API is required.

19. Explicitly Out of Scope for Phase 1

- Automatic document edge detection.
- Auto crop and perspective correction.
- Advanced scan enhancement.
- OCR/text extraction.
- PDF merge/split/compression.
- PDF-to-image.
- Password-protected PDF.
- Cloud backup and multi-device sync.
- Authentication.
- Subscriptions/payments.
- Online conversion APIs.

20. Future Roadmap

Phase	Expansion
Phase 2	Crop, rotate, filters, page size, orientation, margins and quality controls.
Phase 3	PDF history and local document management.
Phase 4	Advanced scanner: edge detection, auto crop, perspective correction and enhancement.
Phase 5	OCR and text extraction.
Phase 6	Merge, split, compress, PDF-to-image and password protection.
Phase 7	Accounts, Firebase cloud backup and cross-device synchronization.

21. Recommended Development Order

- 1. Create Flutter project + DocVault theme.
- 2. Build Splash and Home.

- 3. Add Camera permission and preview.
- 4. Implement multi-page camera capture.
- 5. Add gallery/multi-image picker.
- 6. Create shared camera/gallery image state.
- 7. Build Review screen.
- 8. Add Remove, Add More and reorder.
- 9. Build isolated PDF generation service.
- 10. Implement local PDF saving.
- 11. Build Result screen with Open/Share.
- 12. Add loading, success and error states.
- 13. Test permissions, offline flow, devices and image sizes.
- 14. Polish UI and prepare release APK.

22. Final Phase 1 Architecture

Home → Camera/Gallery → Shared Image State → Review/Reorder → PDF Service → Local Storage → Result → Open/Share

Camera and Gallery are two input providers feeding the same selection state, so PDF generation remains one shared system. Firebase/API/server is not required in Phase 1.

23. Definition of Done

Phase 1 is complete when a fresh user can install DocVault, choose Camera or Gallery, capture/select multiple images, arrange them, generate a valid PDF, save it locally, open it and share it without an account or internet connection, while common permission/storage errors are handled safely.